Polymorphism

Module-2



Dr.M. Sivakumar
Professor & Technical Trainer / CSE
Saveetha School of Engineering, SIMATS
sivakumarm.sse@saveetha.com
+91-9500600868

Polymorphism

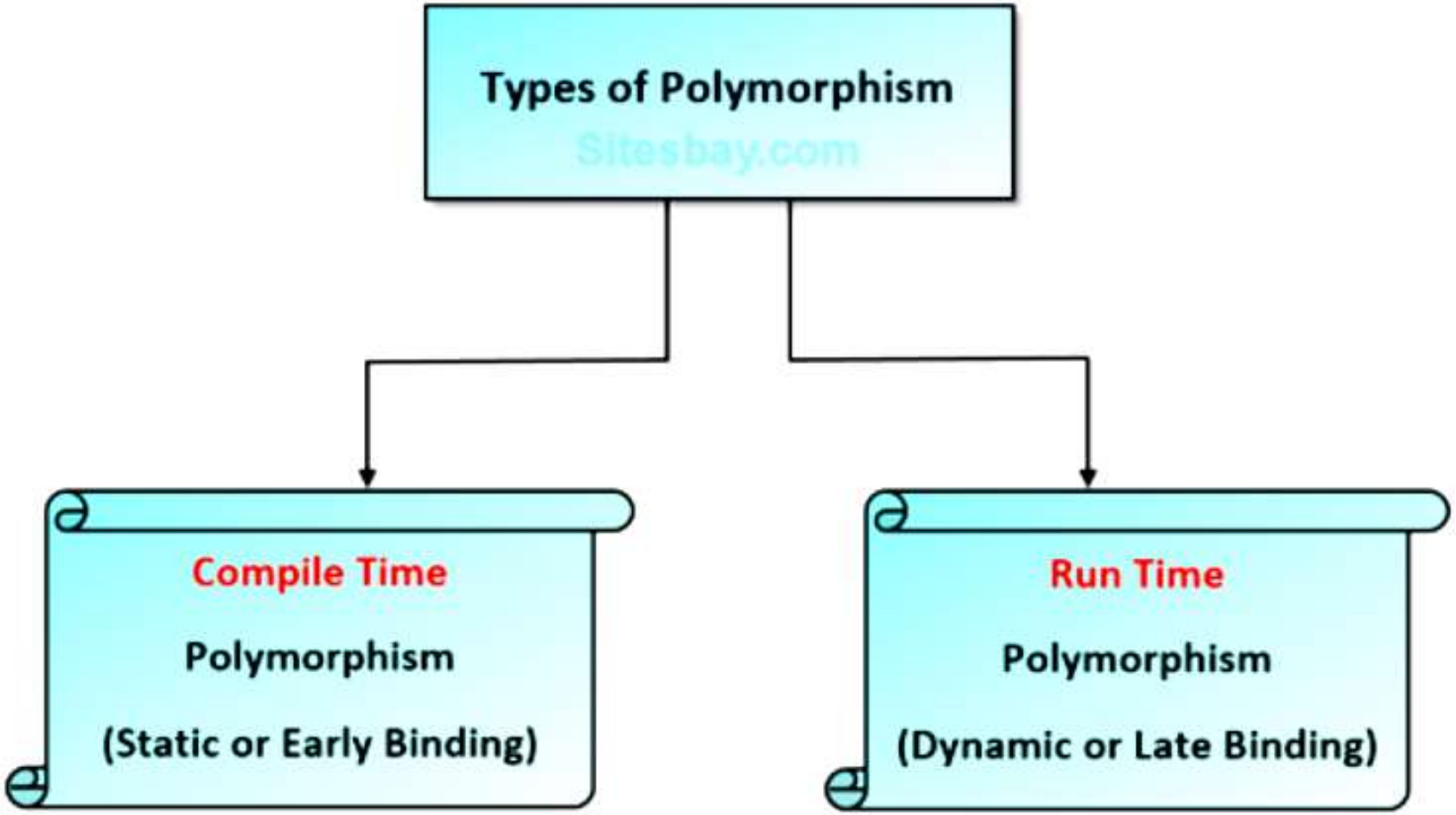
- **Polymorphism** in OOP is the ability of an entity to take several forms. i.e, it refers to the ability of an object (or a reference to an object) to take different forms of objects.

Example: Person



Polymorphism

- Types



1.Compile-time Polymorphism

- As the meaning is implicit, this is used to write the program in such a way, that **flow of control is decided in compile time** itself. It is *achieved using method overloading*.
- When a class has two or more than two methods which are having the same name but different types of order or number of parameters, it is known as **Method overloading.**
- Java allows a function to have the same name if it can distinguish them by their number and type of arguments.

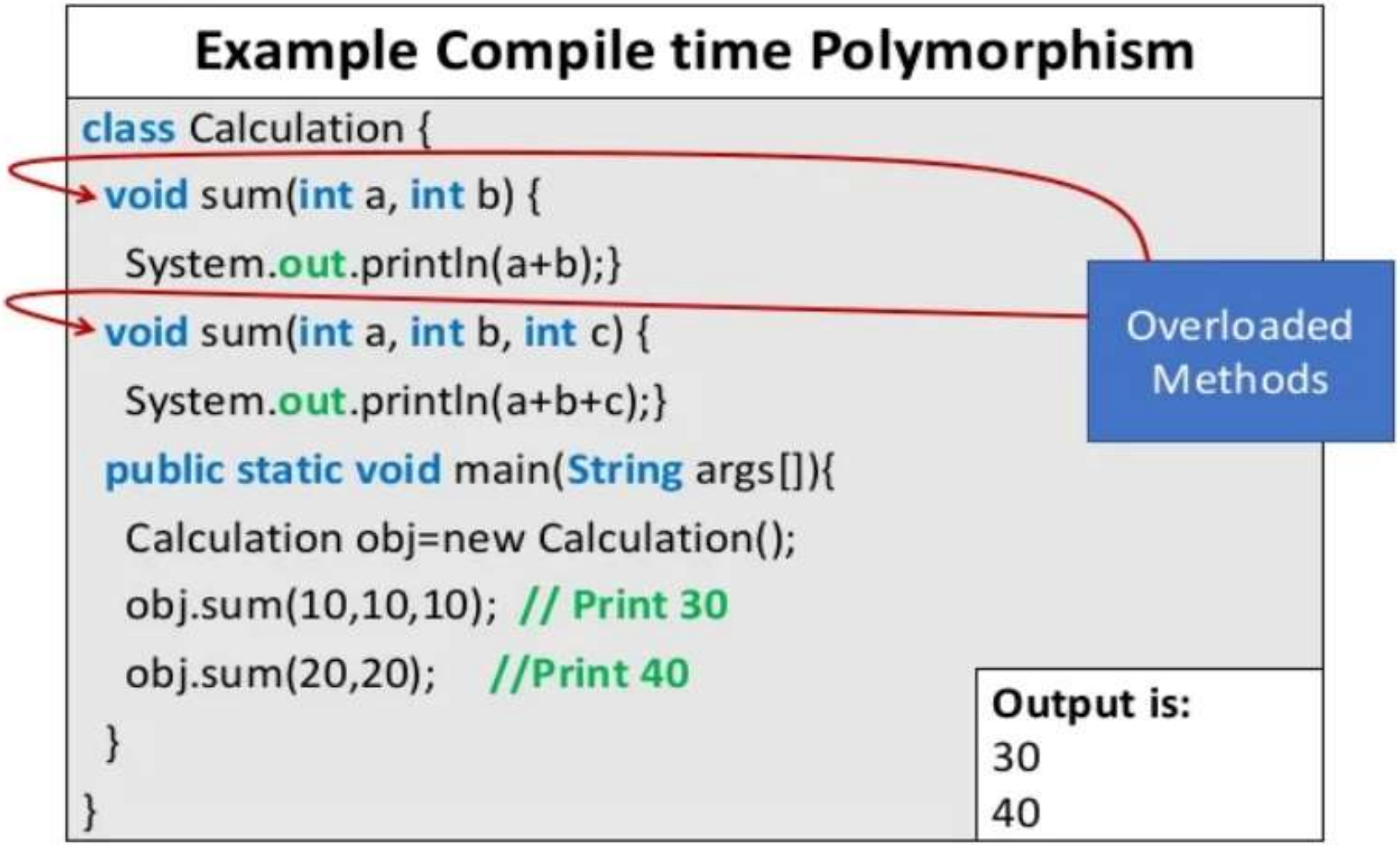
```
1.   float divide(int a, int b){...}

1.   float divide( float x, float y){...}

1.   float divide (float a,int b) {...}
```

1.Compile-time Polymorphism

- **Example**



2.Run-time Polymorphism [Dynamic]

- Dynamic polymorphism is a process in which a call to an overridden method is resolved at runtime, that's why it is called runtime polymorphism.
- Method Overriding is one of the ways to achieve Dynamic Polymorphism.
- Method overriding is a feature which you get when you implement inheritance in your program.

  A simple example can be from real world **e.g. animals.** An application can have Animal class, and its specialized sub classes like Cat and Dog. These subclasses will override the default behavior provided by Animal class + some of its own specific behavior

2.Run-time Polymorphism [Dynamic]

- **Example:**

```
public class Animal {
    public void makeNoise()
    {
        System.out.println("Some sound");
    }
}
class Dog extends Animal{
    public void makeNoise()
    {
        System.out.println("Bark");
    }
}
```

```
class Cat extends Animal{
    public void makeNoise()
    {
        System.out.println("Meawoo");
    }
}
public class Demo
{
    public static void main(String[] args) {
        Animal a1 = new Cat();
        a1.makeNoise();    //Prints Meowoo
        Animal a2 = new Dog();
        a2.makeNoise(); //Prints Bark
    }
}
```

Polymorphism

Advantages

- Method overloading allows methods that perform similar or closely related functions to be accessed through a common name.
- Constructors allowing different ways to initialize objects of a class can implement method overloading.
- Method overriding allows a subclass to use all the general definitions that a superclass provides and add specialized definitions through overridden methods.
- Method overriding works together with inheritance to enable code-reuse of existing classes without the need for re-compilation